

BEGINNER-FRIENDLY PDF EDITION

Testing & Verification

How to verify The Ontology Machine through owner suites, Kernel contracts, frontend checks, database health, artifacts, and installer smoke.

Core rule: test the boundary that owns the risk; then add the smallest broader proof that catches shared-contract and field failures.

Contents

1. Testing & Verification.....	3
2. Verification Philosophy	4
3. Source Of Truth.....	5
4. Test Suite Inventory	6
5. Root Runner	7
6. Test Dojo.....	8
7. Module-Local Runners	9
8. What Each Suite Proves.....	10
9. Kernel Verification.....	15
10. Frontend Verification	16
11. DB Health Checks	17
12. Artifact Tree Checks	19
13. Installer And Runtime Verification.....	21
14. Field-Hardening And Debug-Surgeon Verification	23
15. Known Slow Or Heavy Tests	24
16. Red Run Triage.....	25
17. What A Green Run Proves	26
18. What A Green Run Does Not Prove	27
19. Release Verification Checklist.....	28
20. Verification Command Cookbook	29
21. Operator Rule	31

Recommended pass: read sections 1-4 first, then use the suite, DB, artifact, or installer checklist that matches the change.

1. Testing & Verification

Testing in The Ontology Machine is not one single magic green checkmark. The system is built from owned modules, local runtimes, generated artifacts, SQLite databases, Kernel workflows, agent tool surfaces, and a browser-facing client. Verification follows the same shape.

The important rule is simple: test the boundary that owns the risk. A Corpus Builder schema change should be proven in the Corpus Builder suite first. A Kernel workflow change should be proven in the Kernel suite. A frontend state or progress fix should be proven in the Client Frontend suite. A release build or runtime packaging change should also run the installer and tooling checks.



Figure 1. Minimum verification model: prove the owning boundary first, then add shared-contract and field evidence.

The root runner exists to make that discipline easy without forcing every module into one shared virtual environment.

2. Verification Philosophy

The Machine is not verified by trusting one large end-to-end run. It is verified by stacking smaller proofs:

- module-local tests prove the owner boundary still behaves as documented
- Kernel contract tests prove workflows, state transitions, tool inventories, and owner adapters still agree
- frontend tests prove the GUI can talk to the Kernel, show state, persist config, and render sources without lying to the user
- Corpus Builder tests prove DB schema, materialization, base graph, images, embeddings, merge, and rebuild behavior
- artifact checks prove the folder structure still contains the rebuild evidence needed to recreate a corpus
- DB health checks prove a finished corpus is internally consistent
- installer and runtime checks prove the packaged system can start outside the developer tree
- field-hardening and debug-surgeon runs prove specific field risks after bugs or fragility have been found

A green suite means the known contracts passed. It does not mean every unknown Windows machine, external drive, antivirus lock, slow disk, long path, provider rate limit, or multi-day run has been simulated. That is why this chapter separates automated verification from field-hardening.

3. Source Of Truth

The testing surface is defined by the repo files, not by memory:

- `run-dev-tests.bat` is the root entry point.
- `tools/run-dev-tests.py` discovers suite manifests, orders modules, and dispatches bootstrap and test runs.
- `*/dev-tests/suite.json` files define each module-local suite.
- Python `*/dev-tests/bootstrap.bat` scripts create or refresh the module-local test runtime.
- `Client Frontend\dev-tests\bootstrap.bat` validates bundled Node and local `node_modules`.
- `*/dev-tests/run-tests.bat` runs that module's tests.
- `Client Frontend/dev-tests/run-tests.bat` runs the Node test suite and, in its no-argument form, the installer roundtrip smoke.
- `tools\run-test-dojo.bat` is a planning and inspection surface for future higher-level test orchestration. It is useful context, but it is not the main execution runner yet.
- `pytest.ini` defines repo-level pytest behavior and excludes runtime, package, state, cache, and generated folders from broad test discovery.

Do not infer test ownership from folder names alone. Use the suite manifests and the root runner. Some old state folders and pytest temp folders can contain permission-protected or stale generated files. Blind recursive scans through the whole repo are noisier and less reliable than the manifest-driven runner.

4. Test Suite Inventory

At the time of this handover, the dev-test surface contains roughly 1,224 test files across module and tooling suites.

Suite	Kind	Alias	Approx. test files	Default runner	Main proof
00 - Orchestrator	Python	orchestrator	190	00 - Orchestrator\dev-tests\run-tests.bat	GUI orchestration, artifact setup, pipeline routing, credentials, progress, debug bundles
01 - Optimizer	Python	optimizer	127	01 - Optimizer\dev-tests\run-tests.bat	input profiling, mail/image/PDF handling, OCR, optimizer outputs, runtime packaging
02 - Interpreter	Python	interpreter	53	02 - Interpreter\dev-tests\run-tests.bat	LLM request/response contracts, provider transport, enrichment/interpreter stage behavior
03 - Validator	Python	validator	31	03 - Validator\dev-tests\run-tests.bat	claim validation, table awareness, numeric coercion, safety limits, golden reports
04 - Normalizer	Python	normalizer	96	04 - Normalizer\dev-tests\run-tests.bat	dynamic projection routing, semantic release, materialization, review policy, prompts
05 - Corpus Builder	Python	corpus-builder	114	05 - Corpus Builder\dev-tests\run-tests.bat	DB schema, ingestion, base graph, embeddings, page images, merge, rebuild
06 - Edit Suite	Python	edit_suite	63	06 - Edit Suite\dev-tests\run-tests.bat	editors, owner bundles, form behavior, operation runner, UI workflow
07 - MCP Server	Python	mcp-server	119	07 - MCP Server\dev-tests\run-tests.bat	MCP tool catalog, governance, visibility, handlers, support monitor
08 - Semantic Control Kernel	Python	kernel	253	08 - Semantic Control Kernel\dev-tests\run-tests.bat	Kernel state machine, workflow contracts, agents, owner adapters, merge/rebuild, ontology validation
Client Frontend	Node	frontend	164	Client Frontend\dev-tests\run-tests.bat	config UI, chat server, agents, Kernel bridge, source rendering, progress UI, installer smoke
tools	Python	tooling	14	tools\dev-tests\run-tests.bat	runtime build helpers, portable runtime checks, projection/release ratchets

The exact count will move as the project changes. The important thing is that each suite remains locally owned and runnable without borrowing a hidden global environment.

5. Root Runner

The root runner is the normal entry point for broad verification:

```
run-dev-tests.bat --list
run-dev-tests.bat --all
run-dev-tests.bat --run-only --all
run-dev-tests.bat --bootstrap-only --all
run-dev-tests.bat --module "05 - Corpus Builder"
run-dev-tests.bat --module corpus-builder --run-only
run-dev-tests.bat --module kernel --run-only
run-dev-tests.bat --module frontend --run-only
run-dev-tests.bat --module "08 - Semantic Control Kernel" --module "Client Frontend" --run-only
```

The runner performs these steps:

1. `run-dev-tests.bat` finds a bundled Python runtime from the installed module runtimes.
2. It starts `tools\run-dev-tests.py`.
3. The Python runner discovers `dev-tests\suite.json` manifests.
4. Suites are ordered in the main system order: Orchestrator, Optimizer, Interpreter, Validator, Normalizer, Corpus Builder, Edit Suite, MCP Server, Semantic Control Kernel, Client Frontend, tools.
5. Unless `--run-only` is used, the runner calls each selected suite's `bootstrap.bat`. For Python suites this rebuilds or refreshes `dev-tests\.venv` from the bundled runtime and offline lockfiles or wheelhouses. For Client Frontend this validates bundled Node and `node_modules`.
6. Unless `--bootstrap-only` is used, it calls each selected suite's `run-tests.bat`.

Root runner behavior worth remembering:

- The root command requires `--all`, `--module`, or `--list`.
- `--run-only` assumes the suite environments already exist.
- `--bootstrap-only` is useful after runtime dependency changes.
- Multiple `--module` arguments can be combined.
- Module aliases are accepted where the suite manifest defines them.
- A legacy root `.venv` may be mentioned in warnings, but module suites use their own `dev-tests\.venv` environments.

The root runner is the right choice before a handover, before a release build, after broad refactors, after Kernel workflow changes, and after touching shared contracts.

6. Test Dojo

The repo also contains a Dojo test-planning surface:

```
tools\run-test-doj.bat list
tools\run-test-doj.bat inspect --suite all
tools\run-test-doj.bat run --suite orchestrator-ui
```

This is not the main execution runner. At the time of this handover, Dojo `run` writes skeleton planned reports and `--execute` is not implemented. Treat it as inventory and future orchestration scaffolding, not as proof that tests were executed.

7. Module-Local Runners

For daily work, run the owner suite closest to the change:

```
00 - Orchestrator\dev-tests\run-tests.bat
01 - Optimizer\dev-tests\run-tests.bat
02 - Interpreter\dev-tests\run-tests.bat
03 - Validator\dev-tests\run-tests.bat
04 - Normalizer\dev-tests\run-tests.bat
05 - Corpus Builder\dev-tests\run-tests.bat
06 - Edit Suite\dev-tests\run-tests.bat
07 - MCP Server\dev-tests\run-tests.bat
08 - Semantic Control Kernel\dev-tests\run-tests.bat
Client Frontend\dev-tests\run-tests.bat
tools\dev-tests\run-tests.bat
```

If a local environment is missing or stale, bootstrap it first. Python suite bootstraps create or refresh `dev-tests\.venv`; the Client Frontend bootstrap checks the bundled Node runtime and `node_modules`:

```
05 - Corpus Builder\dev-tests\bootstrap.bat
08 - Semantic Control Kernel\dev-tests\bootstrap.bat
Client Frontend\dev-tests\bootstrap.bat
```

Targeted examples:

```
05 - Corpus Builder\dev-tests\run-tests.bat tests\test_basic_relation_mining.py
08 - Semantic Control Kernel\dev-tests\run-tests.bat
tests\test_phase7_agent_tool_inventory.py
Client Frontend\dev-tests\run-tests.bat dev-tests\tests\main-app-kernel-progress.test.js
```

Most Python module runners call `python -m pytest tests -q`. Targeted file runs are first-class only in runners that switch to `pytest %*` when arguments are provided. Other Python runners append arguments after `tests -q`, so file arguments are not uniformly narrow. The safe pattern is always to run through that module's `dev-tests` runner and check the runner behavior before assuming the selector shape.

Corpus Builder excludes tests marked `stress` by default:

```
05 - Corpus Builder\dev-tests\run-tests.bat
```

That default is intentional. Corpus Builder has heavier DB and artifact tests, and the normal developer run should not accidentally become a long stress run. To run the current stress-marked loader concurrency test directly:

```
cd /d "05 - Corpus Builder\dev-tests"
.venv\python.exe -m pytest tests\test_loader_concurrency.py -q -m stress --basetemp "%TEMP%
\om-cb-pytest-stress"
```

8. What Each Suite Proves

00 - Orchestrator

The Orchestrator suite proves the desktop-facing workflow shell can still build and route pipeline work correctly. It covers app bootstrap, admin/runtime contracts, credentials, debug host behavior, error bundle routing, Kernel artifact tree and batch manifests, model catalog behavior, progress state, manual final notices, pipeline input and output routing, and field-hardening guards around startup and state.

This suite should be run after changes to:

- Orchestrator GUI behavior
- run creation and target selection
- artifact tree creation from the Orchestrator
- credential entry or config persistence
- Kernel bridge calls made from the Orchestrator
- pipeline progress and final notice handling

01 - Optimizer

The Optimizer suite proves raw input files can be profiled, converted, cleaned, and prepared for later extraction. It covers input catalog behavior, file profiling, plugin detection, mail handling, Outlook store parsing, image/PDF processing, OCR request persistence, security cleanup, hash and naming logic, single-file and batch processors, error output, resilience paths, and runtime packaging.

This suite should be run after changes to:

- mail, PDF, image, or scan handling
- OCR request persistence
- optimizer output folder structure
- long filename handling
- input profiling or file type detection
- processor batch behavior

02 - Interpreter

The Interpreter suite proves model-facing extraction requests and stage outputs remain contract-stable. It covers debug bundles, prompts, provider/OAuth transport, batch and single-page workflows, enrichment, interpreter limits, response parsing, validation handoff, runtime paths, and request/response contracts.

This suite should be run after changes to:

- interpreter prompts
- provider request building
- transport and OAuth behavior

- enrichment or interpreter stage limits
- parsed model output contracts
- debug bundle generation

03 - Validator

The Validator suite proves extracted claims are checked against the expected shape before they are normalized. It covers contract/debug behavior, golden reports, file profile and raw claim validation, table profile awareness, numeric coercion, path handling, safety limits, and packaging.

This suite should be run after changes to:

- validation rules
- table or line-item review behavior
- numeric coercion
- golden report expectations
- validator safety limits
- validator output shape used by the Normalizer

04 - Normalizer

The Normalizer suite proves validated extraction can be mapped into the active dynamic projection without falling back to old hardcoded business assumptions. It covers contract/debug stages, surface generation, edit/read bundles, source IDs, dynamic projection routing, semantic release materialization, release merge behavior, prompts, review policy, workflow behavior, and packaging.

This suite should be run after changes to:

- normalizer prompts
- projection guidance
- semantic release creation or merge
- normalized output shape
- review policy
- materialization contracts used by Corpus Builder

05 - Corpus Builder

The Corpus Builder suite proves normalized artifacts can become a corpus DB. It covers schema creation and migration, DB loading, page images, embeddings, document materialization, semantic release attachment, reset/rebuild behavior, merge and additive merge behavior, batch reingest, evidence readers, basic relation mining, source-document grouping, page sequence, document-level classification, and structural base graph relations.

This suite should be run after changes to:

- SQLite schema

- ingestion or rebuild logic
- page image persistence
- embedding generation
- base graph or source document logic
- DB merge behavior
- ontology/base layer preservation during merge

06 - Edit Suite

The Edit Suite suite proves editor-facing tools can load, edit, and save the machine's owned configuration and release artifacts. It covers the registry, owner bundles and contracts for modules, form fields, dual-mode editors, operation runner behavior, embedding actions, merge confirmations, rendering, repository behavior, and UI workflow.

This suite should be run after changes to:

- taxonomy or release editor behavior
- owner bundle schemas
- editor forms and field mapping
- operation runner logic
- merge confirmation UI
- Edit Suite packaging or runtime assumptions

07 - MCP Server

The MCP Server suite proves the tool surface exposed to agents is still governed and safe. It covers agent permissions, catalog governance, host bridge behavior, visibility rules, legacy invisibility, event-scoped recovery, tool handlers for modules, support monitor behavior, storage retention, and line of code governance.

This suite should be run after changes to:

- MCP tool schemas
- agent-facing visibility
- tool permission boundaries
- support monitor behavior
- host bridge calls
- tool handler implementation
- governance ratchets

08 - Semantic Control Kernel

The Kernel suite is the largest suite because the Kernel owns workflow orchestration. It proves the state machine, transition evaluator, dialogs, pending interaction handling, event mirror, progress reporting, recovery, permanent tool surfaces, support surfaces, LLM artifact/retry/provider logic, DB creation paths, custom taxonomy/projection

workflows, manual pipeline control, reset, merge, rebuild, MCP contracts, truth snapshots, ontology patch validation, field-ready atomic behavior, path safety, and background process contracts.

This suite should be run after changes to:

- any Kernel workflow
- Taxonomy Agent tools or prompts
- Kernel-to-owner adapter calls
- DB create/merge/rebuild routes
- manual pipeline control
- Kernel progress, final notices, or dialogs
- ontology validation
- background process or state persistence logic

Client Frontend

The Client Frontend suite proves the user-facing browser and local server still behave correctly. It covers API routes, app paths and config, atomic file behavior, browser helper startup, chat store/history, config UI, OAuth races, secrets handling, corpus path governance, HTTP health routes, main app health display, dialogs, progress boxes, recovery, reset, source viewer behavior, layout, minimal agent behavior, ontology agent behavior, pipeline agent behavior, MCP client behavior, provider model catalog, SQLite lock contention, source rendering, vector codec behavior, and vault handling.

The no-argument frontend runner also performs a deploy-installer roundtrip before the Node tests:

```
Client Frontend\dev-tests\run-tests.bat
```

When a specific Node test is passed as an argument, that installer roundtrip is skipped and only the selected Node test path is run.

This suite should be run after changes to:

- frontend config pages
- chat, source, or page-image rendering
- Kernel progress UI
- agent status displays
- frontend server routes
- DB config persistence
- theme/layout behavior
- installer-facing frontend packaging

tools

The tools suite proves repo-level helper scripts and release ratchets still work. It covers runtime build helpers, portable runtime checks, semantic release helpers, projection/release ratchets, validation wrappers, taxonomy refactor contracts, and locale ratchets.

This suite should be run after changes to:

- runtime build scripts
- installer helper scripts
- repo-level release validation
- locale or taxonomy ratchets
- shared tooling used by multiple modules

9. Kernel Verification

Kernel verification deserves special attention because the Kernel is not just "a backend". It is the deterministic control surface that coordinates agents, workflows, owner modules, user dialogs, and long-running background operations.

The Kernel suite should prove these classes of behavior:

- workflow routes select the correct owner operation
- state transitions are explicit and recoverable
- blocked states preserve a useful diagnostic
- pending user interactions are persisted and resumed safely
- permanent tools remain in the expected inventory
- agent-facing workflow tools match the documented Kernel surface
- owner adapter contracts do not return oversized or unstructured payloads
- manual pipeline runs produce progress and final notices
- merge/rebuild routes preserve corpus state that should survive
- ontology validation catches malformed write batches
- background processes are not killed by short UI expectations
- path checks prevent writes outside the selected artifact tree
- persisted progress events reach `kernel.client_frontend_event_batch.v1`
- cursor skip, overflow replay, active progress-chain replay, and ack failures are handled explicitly
- event mirrors and support surfaces can explain what happened after a run

The default Kernel suite is mostly contract-backed, fake-backed, or fixture-backed. That is deliberate: Kernel tests should not require every owner module to execute expensive real workflows just to prove state-machine logic. Real owner workflow smoke for the default-release paths is opt-in with `KERNEL_REAL_OWNER_SMOKE=1` and requires the relevant Orchestrator, Normalizer, and Corpus Builder runtimes to be available.

For Kernel work, a narrow module-local run is often not enough. Pair the Kernel suite with the owner module that implements the operation:

```
run-dev-tests.bat --module kernel --module corpus-builder --run-only
run-dev-tests.bat --module kernel --module frontend --run-only
run-dev-tests.bat --module kernel --module mcp-server --run-only
```

Use this pairing rule:

- DB create, merge, rebuild, or base graph changes: Kernel + Corpus Builder
- Taxonomy Agent tool inventory changes: Kernel + MCP Server + Frontend
- user-facing progress or final notice changes: Kernel + Frontend
- semantic release workflow changes: Kernel + Normalizer + Corpus Builder
- ontology write validation changes: Kernel + Corpus Builder + Frontend

10. Frontend Verification

Frontend verification is partly about UI and partly about truthfulness. The frontend is where the user sees whether the system is ready, which DB is mounted, what the Kernel is doing, which sources support an answer, and whether an agent is asking for input or has finished a workflow.

The automated frontend suite is Node/JSDOM and contract oriented. It uses bundled Node's test runner and fake Kernel or pipeline agents where needed. It is excellent for state, rendering, route, and contract regressions, but it is not the same thing as launching the live app in a real browser.

Important frontend checks:

- the config server starts and opens the config page
- the chat server starts and opens the main app
- credentials can be pasted with keyboard and context menu
- theme selection does not snap back after a delay
- tabbed config views actually hide and show the intended panels
- model configuration persists
- corpus DB path persists
- the mounted DB status shows LLM readiness, base graph state, and ontology lens count
- Kernel progress boxes remain scrollable on small screens
- long-running workflows show meaningful progress, not just "background process running"
- manual pipeline runs receive a final notice
- source links are reconciled against per-message sources plus the chat source catalog fallback used for restored messages
- unresolved source links are visibly suspicious
- page image viewer and source list render on the correct sides
- Ontology Agent and Query Agent prompts are selectable and persisted
- Query Agent and Ontology Agent tool inventories include the compact `get_document_*` document view family and keep the legacy full read available
- compact document views still produce source payloads that the source list can resolve

Frontend tests cover much of this, but visual layout issues still deserve a browser smoke check after UI changes. Scrollability, responsive fitting, source list placement, page image placement, and real launch behavior should be looked at in the running app, not only in JSDOM.

11. DB Health Checks

Corpus DB verification is separate from "the pipeline finished". A run can finish while still leaving warnings, error cases, missing embeddings, stale relations, or incomplete source-document structure.

Use DB inspection for finished corpora. The exact tables vary by schema version, but newer corpus DBs should normally contain:

- documents
- document_pages
- document_page_images
- source_documents
- source_document_pages
- source_document_classifications
- relations
- ontology_activation
- embedding tables
- ontology lens/node/edge/term/assertion/evidence tables

Useful checks:

```
PRAGMA integrity_check;
PRAGMA foreign_key_check;

SELECT COUNT(*) AS documents FROM documents;
SELECT COUNT(*) AS pages FROM document_pages;
SELECT COUNT(*) AS page_images FROM document_page_images;
SELECT COUNT(*) AS source_documents FROM source_documents;
SELECT COUNT(*) AS source_document_pages FROM source_document_pages;
SELECT COUNT(*) AS base_relations FROM relations;
SELECT COUNT(*) AS classifications FROM source_document_classifications;
SELECT COUNT(*) AS ontology_lenses FROM ontology_lenses;
```

When embeddings are expected:

```
SELECT COUNT(*) AS embedding_chunks FROM embedding_chunks;
SELECT COUNT(*) AS embeddings FROM embeddings;
```

When review flags matter:

```
SELECT COUNT(*) AS docs_needing_review FROM documents WHERE needs_review = 1;
```

When ontology lenses are expected:

```

SELECT COUNT(*) AS active_primary_lenses
FROM ontology_activation
WHERE is_active = 1 AND is_primary = 1;

SELECT l.ontology_id, l.name, l.status, a.is_active, a.is_primary
FROM ontology_lenses AS l
LEFT JOIN ontology_activation AS a ON a.ontology_id = l.ontology_id
ORDER BY l.created_at, l.ontology_id;

```

For source-document coverage:

```

SELECT source_document_id, COUNT(*) AS pages
FROM source_document_pages
GROUP BY source_document_id
ORDER BY source_document_id;

```

For base graph checks after `basic_relation_mining`:

- every materialized page should belong to a source document
- page sequence should be deterministic
- `relations` should only describe the corpus base graph, not ontology-specific interpretations
- source-document classification should be conservative
- ambiguous source-document classification should be marked ambiguous or unresolved instead of invented

For ontology checks:

- ontology lenses should have stable identifiers
- primary lens selection should be explicit
- ontology objects should have required object IDs
- evidence links should reference real DB evidence
- ontology embeddings should not be written without required object IDs
- correction, audit, or review lenses should remain lenses, not mutations of base facts

12. Artifact Tree Checks

The Artifact Tree is the rebuild and evidence surface around the DB. It is not just a scratch folder. The DB may hold direct evidence links and page images, but the artifact tree remains the filesystem record that makes rebuilding, debugging, and human inspection possible.

Start with the canonical folder contract before inspecting evidence:

- Input
- Corpus
- Semantic Release
- Documents
- Documents\logs
- Documents\normalized
- Documents\originals
- Documents\page_images
- Documents\raw_extracts
- Documents\requests
- Documents\structured
- Documents\validation
- Error Cases

For Kernel-created or Kernel-attached trees, exact casing matters and extra authoritative top-level or `Documents` folders should be rejected. For Corpus Builder standalone validation, inspect `missing_paths` and `validation_errors` instead of guessing from a partial folder listing.

A healthy finished tree should make these areas understandable:

- raw or original input files
- optimized document images and intermediate files
- request and response traces for model-facing stages
- OCR requests for scan/image paths
- structured interpreter outputs
- validator reports
- normalized outputs
- active Semantic Release files
- corpus DB under `Corpus`
- error cases with enough context to inspect failures
- batch manifests and logs
- merge/rebuild metadata when the tree was produced by those routes

Checks to perform:

- confirm the active Semantic Release exists where the workflow expects it

- confirm `Input` is not the only place where source evidence exists
- confirm page images exist for page-grounded corpora
- confirm model request traces exist for extraction stages that used models
- confirm normalizer request traces exist when normalization used an LLM
- confirm OCR request traces exist when OCR was used
- confirm error cases are copied with enough context to reproduce or inspect
- confirm merge output did not flatten source trees into confusing nested originals
- confirm generated DB is inside the intended artifact tree
- confirm no stale lock or state file claims a process that is no longer the run owner

Avoid treating generated debug state as a second domain truth. Debug bundles, state snapshots, and logs explain what happened. The artifact tree and DB hold the domain evidence.

13. Installer And Runtime Verification

Installer verification is more than "the installer was built". The packaged Machine has to start on a Windows machine without relying on the developer's global Python, Node, or local temporary state.

Important checks:

- bundled Python runtimes exist for Python modules
- bundled Node runtime exists for the Client Frontend
- launcher scripts point at installed paths
- config server starts
- chat server starts
- browser opens after server readiness, not before
- the Client Frontend has a default corpus DB path if the installer is supposed to provide one
- Orchestrator does not receive stale developer-only sample paths unless that is intentionally part of the package
- uninstall removes installed state without deleting user corpora
- generated shortcuts or launcher entries point to real files
- packaged sample data is included only where intended

The Client Frontend no-argument dev-test runner includes a deploy-installer state and payload roundtrip smoke:

```
Client Frontend\dev-tests\run-tests.bat
```

Tooling tests also cover runtime helpers:

```
tools\dev-tests\run-tests.bat
```

For all-in-one installer staging, verify SampleDB is payload-only, Orchestrator state is not post-install seeded, idle SQLite sidecars are skipped, and non-empty `*.db-wal` sidecars fail staging before target mutation.

After installer changes, automated tests should be paired with a manual smoke:

1. install into a fresh target path
2. start the config frontend
3. start the main frontend
4. confirm the configured root path
5. confirm the configured corpus DB path
6. start the Orchestrator
7. run a small create or inspection workflow

8. uninstall

The frontend installer roundtrip does not replace that manual smoke. It checks the minimal deploy/install payload, state snapshot behavior, bundled port-cleaner presence, and exclusion rules such as module-root `data`. It does not start both frontends in a real browser and it does not uninstall the app.

14. Field-Hardening And Debug-Surgeon Verification

Normal tests prove known contracts. They do not automatically prove field readiness. Field readiness is about behavior on other Windows machines, slow drives, external drives, locked files, missing credentials, long paths, stale process state, interrupted runs, and user environments that do not look like the developer workstation.

Use the debug-surgeon workflow when the system is wrong, flaky, leaking, stuck, racing, or returning inconsistent state. The goal is evidence-first debugging:

- reproduce or localize the failure
- identify the owner boundary
- inspect logs, DB state, and process state
- reduce the fix to the responsible code path
- verify with targeted tests

Use the field-hardening workflow when the code works but is too fragile for real machines. The goal is robustness without piling patches on top:

- tolerate slow disks and antivirus delays
- avoid short hardcoded timeouts for long workflows
- make process state recoverable
- handle locked files and stale state clearly
- keep path handling safe and Windows-aware
- avoid accidental global runtime dependencies
- keep error messages actionable

A hardening pass should be bounded. It should produce one audit, integration and report cycle, rank findings, assign or fix the top risks, record targeted verification, and stop. If more issues are found, record them as follow-up scope instead of letting one run expand forever.

Before non-trivial hardening or debugging work, preserve the Machine's normal owner boundaries, contract entry points, runtime discipline, truth categories, dev-test discipline, and Kernel/Control-Plane responsibilities. If a proposed fix would break those boundaries, report the conflict and propose a smaller owned fix instead of patching locally across modules.

15. Known Slow Or Heavy Tests

Some verification paths are intentionally heavier:

- `run-dev-tests.bat --all` can be slow because it bootstraps and runs every discovered suite.
- `run-dev-tests.bat --run-only --all` is faster only if all environments are already bootstrapped.
- `Client Frontend\dev-tests\run-tests.bat` without arguments runs a deploy+installer state/payload roundtrip smoke before Node tests.
- Corpus Builder stress tests are excluded by default with `-m "not stress"`.
- Kernel tests are numerous because workflows, state transitions, recovery, owner adapters, and agent surfaces are contract-heavy.
- Runtime and installer tests can touch a lot of disk state.
- DB merge/rebuild and ingestion smoke checks can take real time, especially on weaker machines or large corpora.

The default Corpus Builder runner always appends `-m "not stress"`, even when a selector is passed. Use the direct stress command from the module-local section when the stress-marked loader concurrency path is the thing being verified.

For small code edits, run the closest module suite first. For handover, release, DB schema, Kernel workflow, or installer work, run the broader stack.

16. Red Run Triage

When a suite fails, do not immediately patch around the symptom. Identify which boundary is speaking.

Common failure patterns:

- Missing `.venv`: run the module bootstrap or root runner without `--run-only`.
- Missing bundled runtime: verify module runtime folders or run runtime build checks.
- Missing pytest dependency: bootstrap the module suite.
- Node test cannot start: check the bundled Node runtime and frontend dependency state.
- Access denied inside old pytest temp or state folders: avoid blind recursive scans; close processes and clean stale generated state if needed.
- Kernel owner error: inspect the owner adapter response and the owning module logs, not just Kernel state.
- Kernel timeout: check whether the operation is legitimately long-running or whether progress/state stopped updating.
- Oversized owner response: fix the adapter to return compact summaries or artifact references instead of huge payloads.
- SQLite "too many SQL variables": batch the SQL operation.
- SQLite locked: inspect process ownership and retry behavior.
- Path length errors: shorten generated names or reduce nesting at the owner module, not only at the caller.
- Frontend "Failed to fetch": verify whether the chat server, config server, or only one of them is running.
- Source mismatch: inspect both the retrieved source list and the agent's textual citations.
- DB health mismatch: check whether the run was ingestion-only, base graph mining, ontology mining, merge, or rebuild before declaring data missing.

The right fix is usually at the owner boundary where the invalid state was created, not at the later layer that merely reported it.

17. What A Green Run Proves

A green run proves the selected contracts passed:

- module-local regression tests passed
- known schema and artifact assumptions still hold
- Kernel workflow contracts and tested owner adapter contracts still match the selected verification mode
- agent tool inventories still match the expected surfaces
- compact document view tools still route through the shared document repository and do not break source reconciliation
- frontend server and UI behaviors covered by tests still work
- packaged runtime helpers covered by tests still work
- known DB and artifact edge cases remain protected

For the selected scope, this is meaningful evidence. It is not theater.

18. What A Green Run Does Not Prove

A green run does not prove:

- every external provider will behave the same way
- every OAuth or billing condition is covered
- every multi-day run will survive every machine sleep, disk disconnect, or antivirus lock
- every possible corpus will produce a perfect taxonomy
- every generated ontology lens is semantically good
- every user-facing phrase is ideal
- every large merge has been tested at production scale
- every Windows machine will have identical permissions and path behavior
- the installer is release-ready unless the installer path was part of the selected verification

This distinction matters. The Machine is evidence-bound; the test story should be evidence-bound too.

19. Release Verification Checklist

Before a serious handover or release candidate:

1. List discovered suites.

```
run-dev-tests.bat --list
```

2. Run touched module suites.

```
run-dev-tests.bat --module corpus-builder --run-only  
run-dev-tests.bat --module kernel --run-only  
run-dev-tests.bat --module frontend --run-only
```

3. For schema, merge, rebuild, ontology, or embedding work, inspect a real DB.
4. For artifact tree work, inspect the generated tree and error cases.
5. For frontend work, start the app and smoke the changed UI path.
6. For installer work, run the frontend no-argument dev-test runner and a manual install/start/uninstall smoke.
7. Run the full suite when the local environments are ready.

```
run-dev-tests.bat --run-only --all
```

8. If environments are not ready, run the full bootstrap and test pass.

```
run-dev-tests.bat --all
```

9. Record any residual risk in the handover notes instead of pretending the suite proves what it does not cover.

20. Verification Command Cookbook

List suites:

```
run-dev-tests.bat --list
```

Bootstrap all suites:

```
run-dev-tests.bat --bootstrap-only --all
```

Run all already-bootstrapped suites:

```
run-dev-tests.bat --run-only --all
```

Run one suite by display name:

```
run-dev-tests.bat --module "08 - Semantic Control Kernel" --run-only
```

Run one suite by alias:

```
run-dev-tests.bat --module kernel --run-only
```

Run a paired owner check:

```
run-dev-tests.bat --module kernel --module corpus-builder --run-only
```

Run one Python module test file:

```
05 - Corpus Builder\dev-tests\run-tests.bat tests\test_basic_relation_mining.py
```

Run the current Corpus Builder stress-marked path directly:

```
cd /d "05 - Corpus Builder\dev-tests"  
.venv\python.exe -m pytest tests\test_loader_concurrency.py -q -m stress --basetemp "%TEMP%\  
\om-cb-pytest-stress"
```

Run Kernel default-release real owner smoke when the required owner runtimes are available:

```
set KERNEL_REAL_OWNER_SMOKE=1  
08 - Semantic Control Kernel\dev-tests\run-tests.bat  
tests\test_phase9_default_release_paths.py  
set KERNEL_REAL_OWNER_SMOKE=
```

Run one frontend test file:

```
Client Frontend\dev-tests\run-tests.bat dev-tests\tests\main-app-kernel-progress.test.js
```

Run the frontend installer roundtrip and Node suite:

```
Client Frontend\dev-tests\run-tests.bat
```

Run tooling tests:

```
tools\dev-tests\run-tests.bat
```

Inspect Dojo suite inventory:

```
tools\run-test-dojobat list  
tools\run-test-dojobat inspect --suite all
```

Check this documentation file for spelling and whitespace after edits:

```
npx.cmd --no-install cspell --no-progress "The Machine Doku\11_Testing_Verification.md"  
git diff --check -- "The Machine Doku\11_Testing_Verification.md"
```

21. Operator Rule

Do not treat test output as decoration. If a test fails, either fix the owning boundary, defer it with a written reason, or mark it as needing a user decision. Silent red tests rot the handover surface faster than missing tests.